1.Implement an 8-puzzle problem using

i. DFS

```
goal = (1, 2, 3, 4, 5, 6, 7, 8, 0)

def dfs(state, depth, limit, path):
    if state == goal:
        return path

    if depth == limit:
        return None

    zero = state.index(0)
    row, col = zero // 3, zero % 3

    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for dr, dc in moves:
        r, c = row + dr, col + dc

        if 0 <= r < 3 and 0 <= c < 3:
            pos = r * 3 + c
            new_state = list(state)

            new_state[zero], new_state[pos] = (
                new_state[pos], new_state[zero]
            )

            new_state = tuple(new_state)

            if new_state not in path:
                result = dfs(
                    new_state, depth + 1, limit,
                    path + [new_state]
                )

                if result:
                    return result
```

```
    return None


start = tuple(map(int, input("Enter 9 numbers: ").split()))
limit = int(input("Enter depth limit: "))

solution = dfs(start, 0, limit, [start])

if solution:
    print("Solution found:")
    for state in solution:
        for i in range(0, 9, 3):
            print(state[i:i+3])
        print()
else:
    print("Solution not found within depth limit.")
```

Output:

```
Enter 9 numbers: 1 2 3 4 5 6 0 7 8
Enter depth limit: 5
Solution found:
(1, 2, 3)
(4, 5, 6)
(0, 7, 8)

(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```

---

ii. Iterative deepening search (IDS) algorithm

```
goal = (1, 2, 3, 4, 5, 6, 7, 8, 0)


def dfs(state, limit, path):
    if state == goal:
```

```
        return path

    if limit == 0:
        return None

    zero = state.index(0)
    row, col = zero // 3, zero % 3

    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for dr, dc in moves:
        r, c = row + dr, col + dc

        if 0 <= r < 3 and 0 <= c < 3:
            pos = r * 3 + c
            new_state = list(state)

            new_state[zero], new_state[pos] = (
                new_state[pos], new_state[zero]
            )

            new_state = tuple(new_state)

            if new_state not in path:
                result = dfs(
                    new_state, limit - 1, path + [new_state]
                )

                if result is not None:
                    return result

    return None


def ids(start):
    depth = 0

    while True:
        print("Trying depth:", depth)
```

```
        result = dfs(start, depth, [start])

        if result is not None:
            return result

        depth += 1


def display(solution):
    for state in solution:
        for i in range(0, 9, 3):
            print(state[i:i + 3])
        print()


start = tuple(map(int, input("Enter 9 numbers (0 for blank): ").split()))

solution = ids(start)

print("\nSolution found!")
print("Number of moves:", len(solution) - 1)
display(solution)
```

OUTPUT:

```
Enter 9 numbers (0 for blank): 1 2 3 4 5 6 7 0 8
Trying depth: 0
Trying depth: 1

Solution found!
Number of moves: 1
(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```